Security Design

PetPals is a public portfolio demo running on free hosting (Vercel, Render and Supabase, with Groq for the AI features). That shapes what it has to defend against: anyone on the internet can reach it, it must never cost money, and it holds personal-looking data, such as addresses and government IDs, for six roles. This document records each security decision, why it was made, and its known limits. Figures are as of 30 October 2026.

1. Overview

Security is layered, so no single control has to be perfect:

- **Who you are** — short-lived access tokens in memory, rotating hashed refresh tokens in a locked-down cookie, bcrypt passwords, and an account-status check on every request.
- **What you can do** — role checks on every endpoint, ownership and shelter scoping in the services, and a minimal allow-list for workers still awaiting approval.
- **How much you can do** — rate limits per email, per user and per visitor, plus an app-wide daily cap on AI calls, on hosting that can't run up a bill.
- **What is kept and shown** — sensitive fields never leave the server, government IDs sit in a private bucket behind short-lived links, and payments never touch PetPals' servers.

> ***Approach*** *— every protection was checked against the live deployment, not just in tests. Two of the decisions below exist because that testing found something tests alone would have missed.*

2. Authentication and Sessions

Decision	Why
Two-token JWT — a 15-minute access token held only in memory, plus a 7-day refresh token in an `httpOnly`, `Secure`, `SameSite=Strict` cookie	Script on the page can't read the refresh token, and a stolen access token stops working within 15 minutes. `SameSite=Strict` blocks cross-site request forgery.
Refresh tokens rotate and are stored hashed (SHA-256), compared in constant time; signing in replaces any other session's token	A database leak doesn't reveal usable tokens, and a replayed old token fails. SHA-256 rather than bcrypt because the token is already long and random — bcrypt added 3–6 seconds to every page load on Render's free CPU and no extra protection.
Passwords hashed with bcrypt (10 rounds)	Slow by design, so guessing passwords from a leaked hash is expensive.
One password policy, enforced on the server — 8+ characters with an uppercase letter, a lowercase letter, a digit and a symbol — on register, reset and change	The client mirrors it only for live feedback; the server is the authority and answers 422 listing what's missing.
Account status checked on every request, not just at login	A deactivated, banned or closed account loses access straight away, even with an unexpired access token.
Changing or resetting a password signs out every other session	Changing the password is how people lock out someone who got in.
The live notification stream sends the token in the `Authorization` header and closes when it expires	Tokens never appear in URLs, so they don't end up in server or proxy logs.

Decision	Why
The browser reaches the API through the site's own domain (Vercel's `/api` rewrite)	The refresh cookie stays first-party, so it keeps `SameSite=Strict` and survives Safari's third-party cookie blocking.

3. Authorization

Decision	Why
Role-based access on every endpoint (`authenticate` + `authorizeRoles`), checked on the server	Hiding a button in the interface is not access control.
Ownership checks in the services — another user's application, donation, appointment or notification answers exactly like a missing one (404)	Doesn't reveal that the record exists.
Shelter scoping — staff and vets see and change data at their own shelter only	Keeps the shelters in the network isolated from each other.
A minimal allow-list for workers awaiting approval (`authenticate.allowPending`): own profile, government ID, onboarding, logout, email verification, password change	An unapproved staff member, vet or volunteer can finish onboarding, and nothing more.
Approval needs a verified government ID and finished onboarding (409 otherwise)	Nobody joins a shelter team unvetted.
Login redirects only follow paths on this site	Blocks open-redirect phishing such as `/login?redirect=https://evil.example`.

4. Emailed Links and Account Recovery

Decision	Why
Link tokens are random, stored only as a SHA-256 hash, single-use and short-lived — reset 1 hour; verification, reactivation and email change 24 hours. A new link cancels older unused ones	The raw token exists only in the email, so neither a database leak nor an old email grants access.
"Forgot password" and "reactivate" always give the same answer	Nobody can use them to find out which emails have accounts.
A 60-second resend cooldown per user	Stops someone flooding a stranger's inbox.
Email change is two-step — the link goes to the new address and a heads-up goes to the old one	A hijacker can't quietly move the account to their own email.

Decision	Why
Soft verification gate — anyone can sign in unverified, but applying to adopt and donating need a verified email	Payments and applications are tied to an address that has been proven.

5. Abuse and Cost Protection

Decision	Why
Rate limits (`config/rateLimits.js`): login 10 per 15 min per email · sign-up 10 per hour · emailed-link requests 5 per 15 min · link redemption 20 per 15 min · AI 30 per hour per user · the whole API 300 per minute per visitor	Slows password guessing, sign-up spam, inbox flooding and quota burning. Each limit answers 429 in the standard error format, with how long to wait.
Login is limited per email, not per IP	Password guessing is capped even from many addresses, and one person's typos never lock out others on the same network.
The visitor's real IP is read from the proxies' headers — `x-vercel-forwarded-for` through the site, else `cf-connecting-ip` when calling the API directly	On Render, the address Express sees is an internal load balancer that changes on almost every request (see the callout below).
App-wide daily AI cap (`config/aiUsage.js`): 200 provider calls per UTC day across all users	Many throwaway accounts can't drain the free AI quota between them. Past the cap, matches fall back to rule scores and the summary says "try again tomorrow".
$0 by design — free plans only, no payment card on any provider	A traffic spike or abuse can't produce a bill; the worst case is a paused service or used-up free quota.

> ***Found by testing the live site —*** *after the rate limits were deployed, 14 wrong logins in a row were never blocked. A temporary endpoint that echoed the request's addresses showed why: the limiter was counting by Render's internal load balancer, a different one almost every request, so each attempt looked like a new visitor. The fix reads the visitor's address from Vercel's and Cloudflare's headers; re-tested live, the 11th wrong login is now refused.*

6. AI Features

Decision	Why
No free-text input reaches the model — the matcher sends fixed quiz wording (the server maps option codes to text) and staff-written pet descriptions; the summary sends health records	The AI can't be used as a general chatbot on the project's quota, and users can't write instructions into the prompt.
Prompt-injection guard — the prompt tells the model that pet descriptions, record notes and transfer reasons are data, never instructions	Staff-written free text can't steer the model.
No personal data is sent — no adopter details, and no people's names in passport summaries	Keeps what reaches a third-party provider to a minimum.

Decision	Why
Structured output, re-checked — strict JSON schema; unknown or repeated IDs dropped, scores clamped to 0–100, text cut to fit; summary points must cite a real record	A misbehaving model can't write invalid data.
Results are cached — matches until the quiz, profile or pets change; summaries by a hash of their input	Most requests never call the provider, which limits cost and exposure.
Graceful fallback — any AI failure gives rule-only matches, or a summary error that is never cached	The site works fully with AI switched off or failing.
Medical judgement stays rule-based — overdue vaccines come from the rules, and the model is told not to judge due dates	The AI summarises; it never decides anything clinical.

7. Data Protection

Decision	Why
Sensitive fields never leave the server — passwords, refresh tokens, Stripe IDs	Left out of every response shape.
Government IDs live in a private storage bucket, shown only through 5-minute signed links, with the ID number masked (`*****4567`) in every response	Even approvers never get a permanent link or the full number.
Uploads are checked on the server — 5 MB maximum, one file, JPEG, PNG, WebP, HEIC or PDF only	No arbitrary file types.
The Supabase service-role key stays on the server	The browser never holds a key that can bypass storage rules.
Deleting an account removes the person and their ID file but keeps shelter history — appointments, donations and records stay, with the link to the person cleared	Honours deletion without corrupting other people's records.
Prisma for every query; raw SQL only for location search, through parameterised templates	Blocks SQL injection.
Strict validation of fixed-value inputs (400 on an unknown status or filter value)	Unexpected input is rejected at the edge.

8. Payments

Decision	Why
Stripe Checkout (Stripe's hosted page) for application fees and donations	Card details never touch PetPals' servers.
The webhook verifies Stripe's signature on the raw request, and records are created only from a verified `checkout.session.completed`	A forged "paid" event is rejected, and the success page in the browser never creates a record.
Idempotent — the Checkout session ID is unique in the database	A replayed webhook can't record a payment twice.
Production has its own webhook secret, separate from the local development one	Each environment only accepts events signed for it.

> ***Found by testing*** *— the local development webhook secret had first been pasted into the production settings. Signing a dummy event with the local secret and seeing production accept it exposed the mistake before any real payment depended on it.*

9. Infrastructure

Decision	Why
`helmet` **security headers**, with the API docs page served without inline scripts	Standard browser-side hardening; the content-security policy holds on every page.
CORS allows only configured origins (`CORS_ORIGINS`, else `CLIENT_URL`), with credentials	Other sites can't call the API with a user's cookie.
`trust proxy` **set for the hosting proxy**	Correct HTTPS detection, so `Secure` cookies work behind Render.
Secrets only in environment variables — `.env` is ignored by git and kept out of both Docker images; Render generates `JWT_SECRET`; the server's production dependencies have no `npm audit` advisories	No secrets in the repository or the images.
Database reached through Supabase's Session pooler	The direct host is IPv6-only, which Render can't reach; session mode keeps migrations working.

10. The Public Demo

Decision	Why
Worker demo passwords made private — the seeded Admin, Staff and Vet passwords (public in the repository) were changed on the live database by a one-off script that was never committed, and every session for those accounts was signed out	Anyone reading the repository could otherwise sign in as Admin on the live site. Adopter, Volunteer and Donor stay public demo logins.

Decision	Why
"Portfolio demo" notices on sign-up and on every government-ID upload	Asks visitors for made-up details and sample documents, never real ones.
Seeded accounts use fake `@petpals.com` addresses	Real email is never sent to strangers.
Shared demo logins can't be taken over — the Adopter, Volunteer and Donor demo accounts refuse a password change, an email change and closing or deleting the account (409, reason demo_account), a forgot-password request for them sends nothing, and no email is ever sent to their seeded @petpals.com addresses	Their passwords are public, so otherwise one visitor could change the password or email, or delete the account, and lock every later visitor out; and those addresses aren't real PetPals inboxes, so a visitor's actions mustn't email whoever owns the domain. Everything else works as for any account.

11. Known Limits

These are accepted for a demo and would be the next steps for a production service:

- **Counters live in memory on one server.** The rate limits and the daily AI cap reset when the server restarts (a deploy, or waking after 15 idle minutes). Running several servers would need a shared store such as Redis.
- **Per-visitor limits can be dodged by calling the API directly** with a made-up `x-vercel-forwarded-for` header. The limits that matter most — login per email and AI per user — don't depend on it.
- **Deliberate lockouts.** Ten wrong passwords lock that one email out for 15 minutes, which someone could do on purpose — the shared demo logins included.
- **Production and development share one database.** Test runs briefly show test data on the live site.